Contents

Technical Q&A: how it is implemented **1**
1. How is a specimen's orientation measured? 1
2. Where does the 55.6 degree threshold come from? 2
3. What exactly does placement curation measure, and why is it not orientation? . . 2
4. How is the 180 degree cohort identified? 3
5. How does the pool become a draw? 3
6. How is the sampling made reproducible and paired? 3
7. Why round-robin rather than stratified random sampling? 4
8. How is the orientation-diverse draw constructed? 4
9. Is any of this leaking test information? 4
10. How do you stop two incompatible arms being compared? 4
10b. How is mIoU computed here, exactly? 5
11. How is TTA implemented, and how is it kept honest? 5
12. What is actually recorded per run? 6
Short answers, if the question is asked in passing 6

Technical Q&A: how it is implemented

Answers for the "how did you actually implement this" questions, at the level of functions, formulas, thresholds and guards. Ordered so the first eight sections cover canonicalization, which is what backup slides 32 to 34 show. Section 10b covers the metric on slide 3. Every claim here names the file it lives in, so you can open the source if pressed.

How deltas are quoted. Every gain appears twice: in mIoU points, and as a percentage of the arm it was added to. The percentage is the honest comparator across datasets, because a point is worth more at 55 than at 77. So orientation TTA's +5.44 on butterfly is +7.6% of the 71.78 it was added to, while the loss stack's +1.59 on SAM 3 beetle is +2.4% of 66.33. Both readings come from the same paired per-seed measurement; nothing is rescaled.

Source layout referenced throughout:

```
src/occl/curation.py     the criteria, the pools, the draws
src/occl/seeding.py      seeds, prefix nesting, support-set hashing
src/occl/adapters/tta.py orientation TTA
src/occl/finetune/       the cycle loss and its arms
src/occl/registry.py     comparability fields and paired comparison
tests/test_curation.py   the guards named below
```

---

1. How is a specimen's orientation measured?

`curation.body_axis_angle(labelmap, head_id, tail_id)`.

Take the centroid of the head trait and the centroid of the tail trait, then the signed angle of the vector between them:

```
hy, hx = np.nonzero(labelmap == head_id)
ty, tx = np.nonzero(labelmap == tail_id)
angle = np.degrees(np.arctan2(ty.mean() - hy.mean(), tx.mean() - hx.mean()))
```

Returns None if either part is absent, and that specimen is rejected by the `no_axis` counter rather than silently treated as canonical.

Why a directed trait-pair vector and not PCA on the silhouette? Because the failure being filtered is a 180 degree inversion, and a principal axis is undirected: a specimen and its inversion have the same principal axis, so PCA cannot see the defect at all. The directed vector differs by exactly 180 degrees between them. There is also a measurement reason: on spread butterfly wings the silhouette is nearly isotropic, so a principal axis is unstable. Measured with the trait-pair vector, butterfly orientation is extremely well defined (R between 0.993 and

0.9996, circular std 1.6 to 6.9 degrees), which is how we established that butterflies are not mis-oriented and need a different criterion.

Which two traits? The dataset declares them as `axis_parts`. Datasets without that attribute skip the criterion entirely, and the bundle records `criteria_applied` so a skipped criterion is visible rather than implied.

---

2. Where does the 55.6 degree threshold come from?

It is the training pool's own circular standard deviation, computed on the same angles the filter is about to test:

```
R      = hypot(mean(sin θ), mean(cos θ))      # resultant length, 0 to 1
std    = degrees(sqrt(-2 * ln R))             # circular std
accept = angular_deviation(angle, circular_mean(train)) <= std
```

`circular_mean` and `circular_resultant` are used rather than an arithmetic mean because angles wrap: the arithmetic mean of -179 and +179 is 0, which is the opposite direction from the truth. `angular_deviation` wraps the difference into [0, 180].

For beetle this evaluates to 55.6 degrees. It is a property of how the specimens happen to be pinned, not a number chosen to hit a target. Two things enforce that:

- `tests/test_curation.py::test_threshold_is_the_a_priori_training_circular_std` pins the rule, so a later edit cannot quietly substitute a tuned constant.
- The bundle records `threshold_deg`, `train_circular_mean_deg`, `train_resultant_R` and a literal `threshold_rule` string, so an auditor can recompute it.

If the arm had missed its target, the project's stated position was to report the miss rather than move the threshold.

---

3. What exactly does placement curation measure, and why is it not orientation?

`curation.trait_centroids` plus `curation.placement_displacement`.

1. For each specimen, compute the centroid of every present trait.
2. Build a reference configuration from the training split: the **median** x and median y per trait, taken independently. Median rather than mean so a handful of badly placed specimens cannot drag the reference toward themselves.
3. For a candidate, take the mean Euclidean distance between its centroids and the reference's, over the traits the two configurations share. Returns None if they share none.
4. Accept if that displacement is at most `train mean + 1 train std` of the same quantity. Same discipline as the orientation threshold: derived from the training distribution, recorded in the bundle as `placement_threshold_px`.

Why a separate criterion? A specimen can be perfectly upright and still be in the wrong corner of the sheet. Angular deviation is zero in that case and displacement is large, and it is the displacement that breaks mask propagation, because the tracker carries the reference's spatial layout onto the query. The numbers: on cyrbia the correlation between run mIoU and support-query displacement is -0.988, against the -0.710 orientation signal that motivated the beetle arm.

Why butterflies need it and beetles do not. Cambridge butterfly frames are dissected wings laid loosely on a sheet. Cyrbia's trait centroids scatter with a 219.4 px standard deviation against 36 to 60 px for the other four subspecies, and splitting cyrbia by background reveals two imaging protocols filed under one name: green-background frames scatter the wings at 209.9 px, neutral frames sit at 27.7 px like everything else.

---

4. How is the 180 degree cohort identified?

`curation.orientation_of(centroids, reference, size)`, used to build strata rather than to filter.

Compare the candidate's centroid configuration against two things: the reference configuration, and the reference **mirrored about the image centre** (`size - x`, `size - y`). Whichever is closer in mean distance wins, giving `"upright"` or `"rot180"`.

One implementation detail worth stating if asked, because it was a real bug: `size` is the mask's own extent, `max(lm.shape)`, not a hardcoded 1024. Butterfly masks are natively 1024 square, but beetle crops are 172 to 431 px. With 1024 hardcoded, the mirrored reference lands outside the image for beetle and every specimen scores as upright, turning the criterion into a silent no-op. Pinned by `test_orientation_is_measured_against_the_masks_own_extent`.

The reference is built from the **training** split only, and the whole map is memoised per dataset instance because `orientation_diverse_supports` calls it once per run with an identical pool. Without the memo that is roughly 15 minutes of single-threaded mask reading per stage on Butterfly Major.

---

5. How does the pool become a draw?

```
canonical_pool(ds, group, criteria)   ->  (pool_ids, stats)     train masks only
draw_supports(ds, group, pool, selection, shot, run_idx)         single dispatch
  |- canonical_balanced      -> stratified_supports(...)
  |- orientation_diverse     -> orientation_diverse_supports(...)
  '- everything else         -> seeding.sample_supports(...)
```

`canonical_pool` applies completeness first (cheapest), then orientation, then placement, counting rejections by reason. It raises if the pool comes out empty rather than falling back to the unfiltered split.

`draw_supports` exists because this dispatch was duplicated in three places and the copies drifted on the `orientation_diverse` arm: `protocol.build_run` used the orientation-diverse draw while `build_cache.required_butterfly` and `evaluator.distinct_images` fell through to uniform sampling. The consequences, in the order they bite: the cache is pre-built from a support set the run never asks for; the preflight reports the wrong images as missing, so topping up from its own error message does not fix it; and `precompute` warms embeddings no episode uses while every real support misses the cache.

---

6. How is the sampling made reproducible and paired?

`seeding.derive_seed(dataset, group, shot, run_idx)` is a blake2b digest of a human-readable key string, reduced to 31 bits. No global RNG state is involved anywhere.

Two properties do the work:

- **Model independence.** The seed depends only on dataset, group and run index, never on the adapter, dtype or device. Run 7 of Butterfly Major draws the same reference for pretrained SAM 2, every OC-CCL variant, and SAM 3. That is what makes the comparisons paired, which matters because support difficulty is the dominant variance component and cancels exactly when you difference per seed.
- **Prefix nesting.** `sample_supports` passes a literal `0` in the seed's `shot` field and slices `[:shot]` from one permutation, so the 20 one-shot references are exactly the first elements of the 20 five-shot sets. The 1-shot to 5-shot comparison is therefore paired too. Pinned by `test_balanced_selection_preserves_prefix_nesting`.

`sorted()` before permuting is load-bearing: filesystem ordering is not stable across machines.

`seeding.support_set_hash` hashes the **resolved** IDs per run, not the seed, because two runs can share a seed and still draw different references if one image failed to download and was filtered from `train_ids`. That hash gates whether two results may appear in the same table.

---

7. Why round-robin rather than stratified random sampling?

`curation.stratified_supports`. Run `r` takes stratum `(r + k) mod S` for the k-th reference, and the member within that stratum is drawn by the run's seed.

The non-obvious part: stratified **random** sampling does not fix the coverage problem at this sample size. Twenty independent draws over twelve species is a coupon-collector problem, and measured, it still left three species unused, 9 of 12. Equal probability is not equal representation at n = 20. Round-robin gives 12 of 12 with usage 1 to 2, keeps prefix nesting, and makes a 5-shot draw span five distinct species.

It raises if `shot` exceeds the number of strata rather than silently repeating one.

Guard: `test_round_robin_covers_every_species_where_random_sampling_does_not`.

---

8. How is the orientation-diverse draw constructed?

`curation.orientation_diverse_supports`. Split the pool by the stratum map from section 4, then take `n_rot = round(shot * rotated_fraction_of_pool)`, clamped to `[1, shot - 1]`, so both orientations are always represented at 5-shot. Members within each orientation are drawn by the run's seed in a deterministic class order, and the set is topped up from the whole pool if a class is short, keeping the count exact.

The proportion follows the **train** prior (cyrbia: 34 percent rotated, so 2 of 5), never the test distribution, which would be leakage. Where a group has only one orientation the function degenerates to the ordinary draw, so it is a no-op everywhere except cyrbia.

---

9. Is any of this leaking test information?

Three separate answers:

- Every reference criterion is computed from `ds.train_ids(group)`. Never test.
- `test_pool_is_estimated_from_train_only` perturbs `ds.test_ids` and asserts the pool is unchanged. Note where the memoisation lives: `support_pool` caches, `canonical_pool` stays pure, precisely so that the leakage test calls an uncached function and cannot pass vacuously.
- Estimating on train costs nothing anyway: train circular mean 67.1 degrees against test 56.8, concentration R 0.625 against 0.614.

The uncomfortable part of the accounting is not leakage but **label budget**: the best butterfly arm reads about 1,700 training masks to choose one reference, against a headline of one labelled image. The defensible framing is that curation stands in for a human curator's judgement and a human does not need 1,700 masks; the test is to shrink the pool until the drawn set stops changing. That is written down as future work, not claimed.

---

10. How do you stop two incompatible arms being compared?

`registry.COMPARABILITY_FIELDS` includes `protocol.support_selection`, so `registry.compare()` raises on a uniform-versus-canonical pair: they draw different references by construction, and differencing them measures the draw rather than the technique. `data.interp`, `data.cache_format` and `model.tta` are in the same set, each added after a near miss where an arm would otherwise have been classified as comparable.

For the cross-family comparisons in the model swap, there is exactly one sanctioned relaxation: `compare(..., ignore_fields=("model.family", "model.image_size"))`, which pins `support_set_hash` equality and embeds the ignored fields and both their values in every row it produces. A cross-family number that cannot say it is one does not appear.

Guards: `test_registry_refuses_to_pair_the_two_canonical_arms`, `test_registry_refuses_to_pair`

10b. How is mIoU computed here, exactly?

`metrics.score_episode` is the only entry point, and nothing else in the package computes IoU. The order of operations is fixed and enforced structurally: **resize logits, then argmax, then IoU**. Argmax first would nearest-neighbour a label map and change the answer.

1. Resize the (C, H, W) logit stack bilinearly to the ground truth's own resolution (`iou_space="gt_native"`, the default). The alternative, resizing the label map up to the model's resolution, is available as `"model"` for repo compatibility and is not what we report.
2. `logits_to_labelmap` gives each pixel its highest-scoring trait, and background where that best logit is at most 0. That is the paper's "highest score label" overlap rule, and it is equivalent to appending a constant-zero background channel without materialising it. It holds because SAM 2 and SAM 3 mask logits are calibrated so that greater than zero means inside the object. Ties go to the lowest channel index, so the result is deterministic given a fixed `class_ids` order.
3. `per_image_miou` takes IoU per trait **present in this image's ground truth**, then the unweighted mean over those traits. Background is never a term.
4. The run score is the mean over images; the reported number is the mean over 20 runs, with the spread reported beside it.

Semantically: 1.00 means every pixel of every trait is correct. Because the mean is over traits rather than pixels, a hair-thin antenna counts as much as the elytra, which is why beetle scores lower than its silhouette quality suggests.

Three edge cases, each decided deliberately:

- A trait predicted but **absent from the ground truth** contributes no term. It still hurts indirectly, because argmax gave it pixels that present traits wanted.
- A trait **present in the ground truth but never prompted**, because the drawn reference did not contain it, scores 0 and is counted. That is a real failure of the one-shot draw and a main driver of the run-to-run spread. Excluding it would shrink our spread below the paper's.
- A trait in the reference but **empty in this ground truth** is excluded, symmetric with the first case.

`iou_binary`, the merged-foreground IoU, is computed alongside for continuity with the upstream harness, but it is not the paper's metric and is never the headline. It is what the talk calls the silhouette score.

Worked example, slide 3, from `s4c.zip`'s `per_query.csv`, beetle `impunctatus/beetle_1166_231`, run 0: elytra 0.925, pronotum 0.916, head 0.802, legs 0.630, antenna 0.000. The image scores the mean of those five, 0.655. The antenna alone costs 0.13.

---

11. How is TTA implemented, and how is it kept honest?

`adapters/tta.py`, a wrapper around the tracker adapter rather than a modification of it, so every previously recorded number stays byte-stable.

- `rotate_episode` applies `np.rot90(x, 2)` to the reference image and its labelmap together, leaves the query untouched, and suffixes the support IDs with `@r180`.
- The suffix is the embedding-cache key. Without it the second pass is served the upright image's features, silently. `test_tta_precompute_caches_both_orientations` pins it.

- The decision is `confidence(logits) = top[top > 0].mean()` where `top = logits.max(0)`, returning `-inf` when nothing is claimed so an empty prediction always loses. No ground truth is consulted, which is what keeps the protocol one-shot. `test_tta_picks_the_more_confident_orientation_without_ground_truth` pins that.
- `precompute` is served dynamically through `__getattr__` rather than declared as a method, so `hasattr(wrapper, "precompute")` mirrors the inner adapter exactly. A class-level method made the wrapper always claim the capability, which crashed on an adapter without one.

---

12. What is actually recorded per run?

Every bundle carries the curation `stats` verbatim: `criteria`, `criteria_applied`, `pool_size`, `train_size`, `rejected` counts by reason, `axis_parts`, `expected_traits`, `threshold_deg`, `train_circular_mean_deg`, `train_resultant_R`, `threshold_rule`, and for butterfly `placement_threshold_px` and `train_placement_mean_px`.

The point of recording the rule string as well as the value is that the approximation stays auditable after the fact. Curation approximates a human judgement the source paper never defines, so all three support arms are always reported rather than only the curated one.

Short answers, if the question is asked in passing

- **“Is the threshold tuned?”** No. It is the training pool’s own circular standard deviation, recorded per run and pinned by a test.
- **“Why not PCA for the axis?”** A principal axis is undirected and cannot distinguish a specimen from its 180 degree rotation, which is the exact defect being filtered.
- **“Why median for the placement reference?”** So a few badly placed specimens cannot drag the reference toward themselves.
- **“Does curation touch the test split?”** No, and a test asserts the pool is unchanged when the test split is perturbed.
- **“How do you know the 20 draws are the same across models?”** The seed is a hash of dataset, group and run index only, and the resolved IDs are hashed per run into `support_set_hash`, which gates pairing.
- **“Does TTA see labels?”** No. The choice is the mean top logit over claimed pixels.
- **“Is mIoU averaged over pixels or traits?”** Traits, unweighted, over the traits present in each image’s ground truth. A missed antenna costs as much as a missed elytra.
- **“What happens to a trait the reference never showed the model?”** It scores 0 and is counted, because that is a real failure of the one-shot draw.